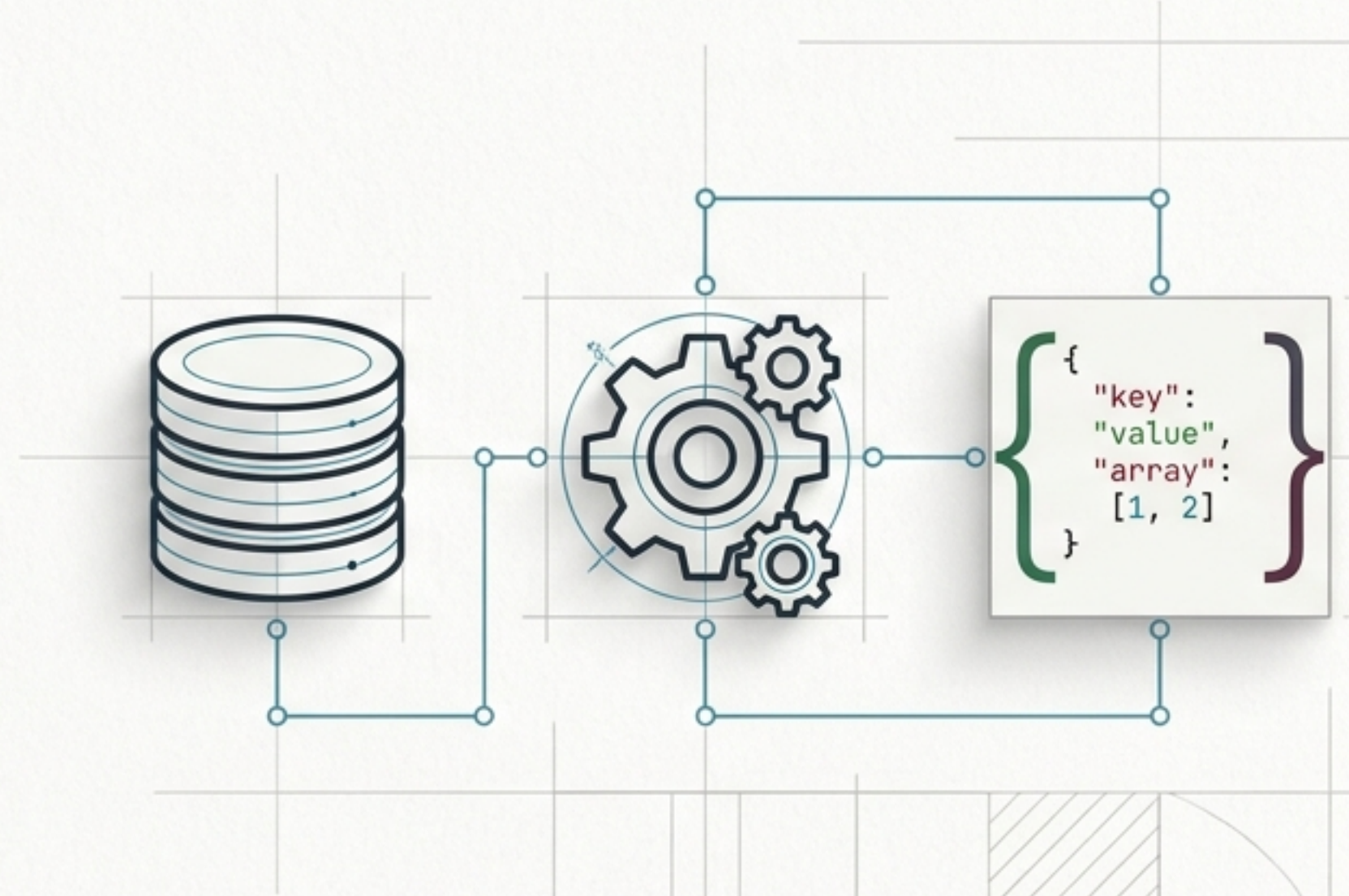


[Status: Online] | Porta: 3000

Arquitetura de API Restful

Construindo o backend do
Nutri_DB com **Node.js** e MySQL



O fluxo da verdade matemática

Validação de Entrada

Coleta **peso** e **altura**. Não realiza cálculos definitivos.
Regra de ouro: Nunca confie nos dados do usuário.

Peso (kg)

Ex: 70.5

Altura (cm)

Ex: 175

ENVIAR

Processamento Seguro

Recebe dados brutos, calcula o **IMC** real e define o **status**.
Protege o sistema contra manipulação de dados sensíveis.



Armazenamento Definitivo

Guarda a verdade inalterável de forma persistente e estruturada no esquema **Nutri_DB**.



Nutri_DB

O Front-end sugere.
O Back-end garante e assina a integridade.

Modelagem da Tabela "clientes"

Esquema: Nutri_DB | Tabela: clientes

id_cliente [PK, Auto Increment]

nome [VARCHAR]

altura [DECIMAL 4,2]

peso [DECIMAL 4,2]

imc [DECIMAL 4,2]

time_stamp [CURRENT_TIMESTAMP]

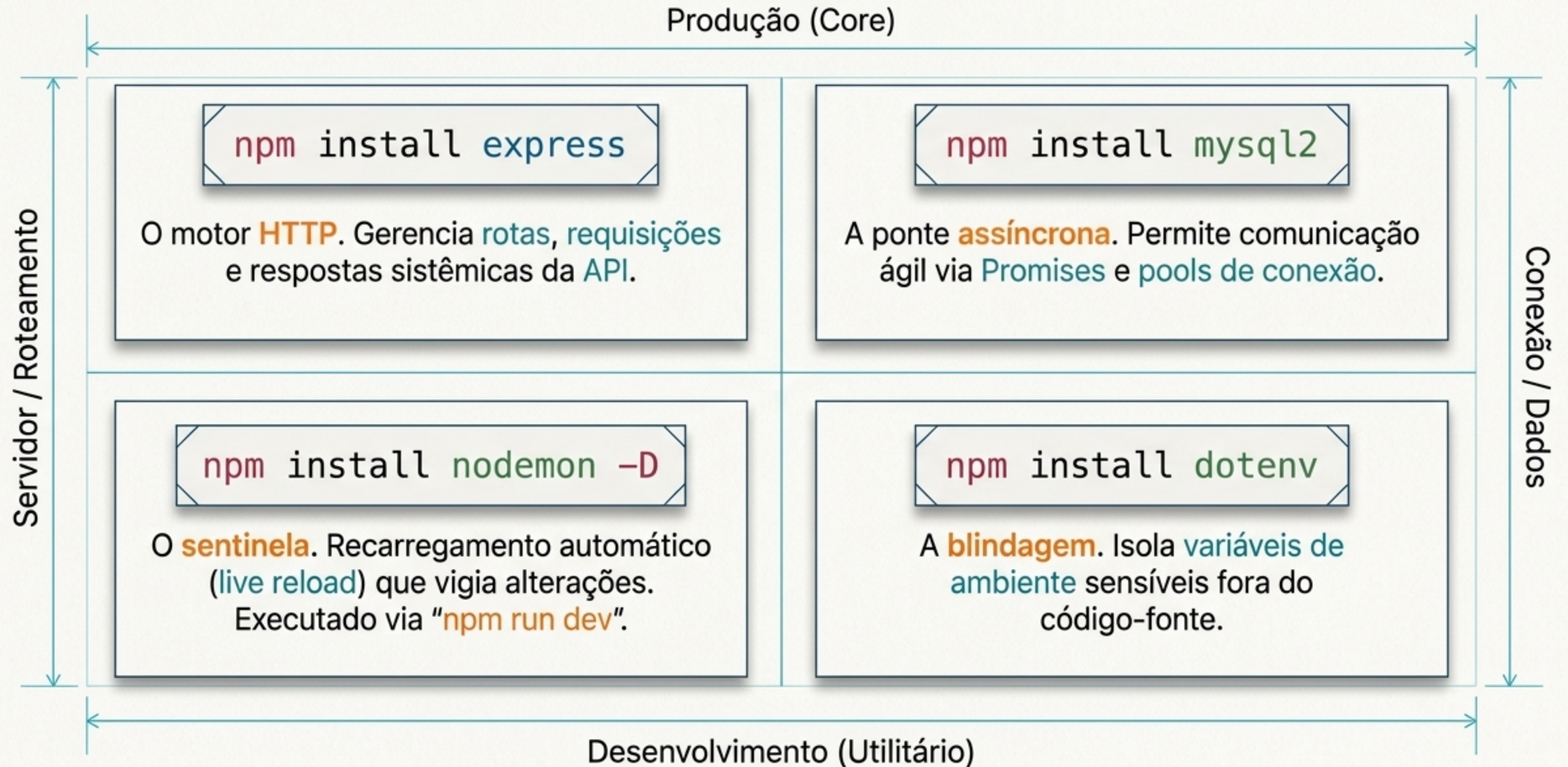
Precisão Absoluta

A escolha por DECIMAL(4,2) em vez de FLOAT garante exatidão em cálculos médicos, permitindo armazenar de 45.00 kg até 104.20 kg sem arredondamentos corrompidos.

Auditoria Contínua

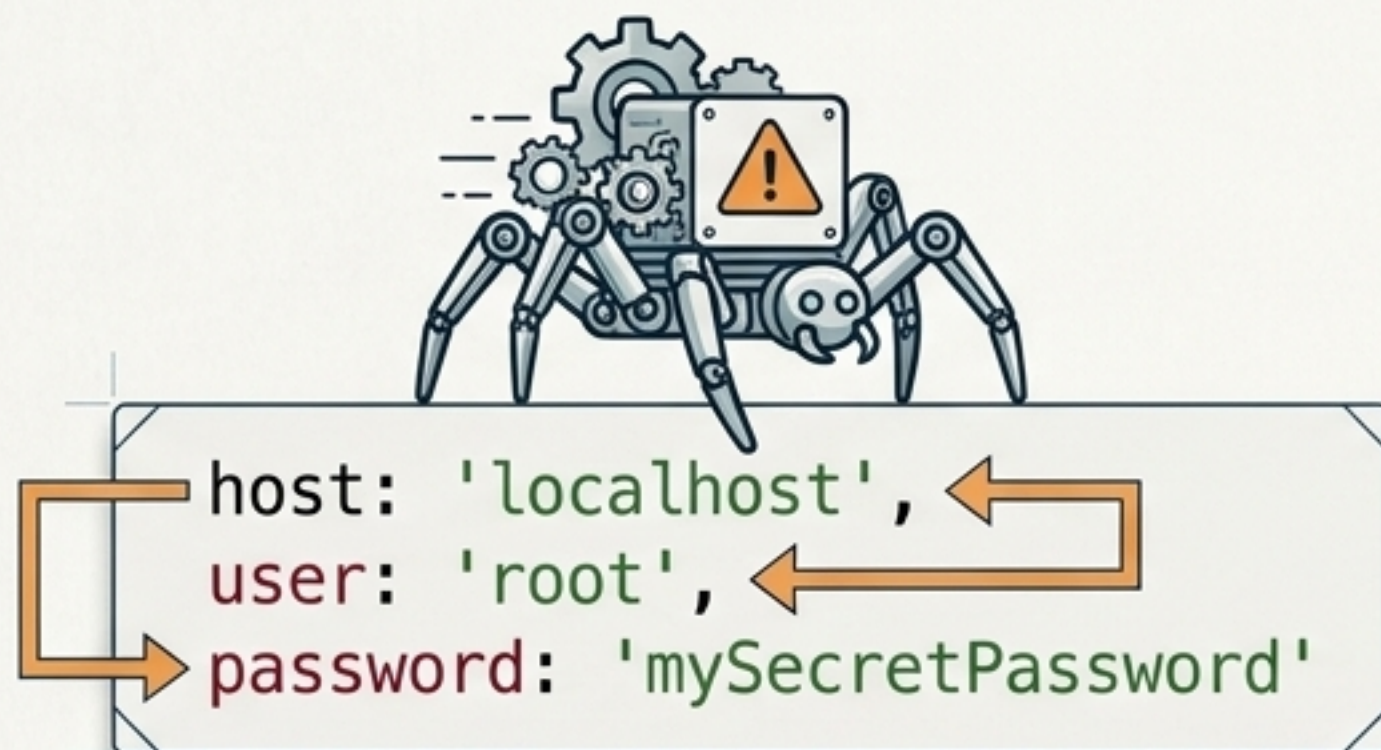
CURRENT_TIMESTAMP registra o momento exato da inserção no banco de forma autônoma, delegando o controle de tempo ao banco e não ao código.

O ecossistema de dependências



O escudo contra robôs mineradores

Credenciais Expostas

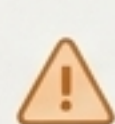


Robôs fazem crawling em plataformas como GitHub buscando chaves em texto plano para invasão de infraestrutura.

Variáveis de Ambiente



Chaves são movidas para o arquivo local .env.
O arquivo .gitignore bloqueia e oculta o .env do controle de versão público.

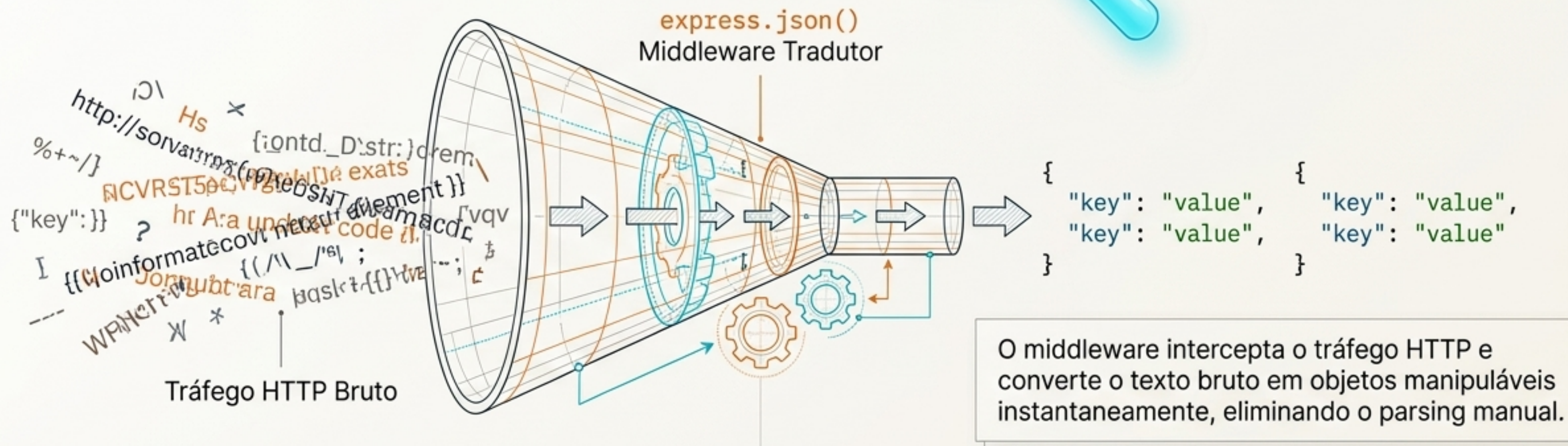


Atenção: Alterações no .env exigem reinicialização manual do servidor.
O Nodemon ignora estas atualizações por design de segurança.

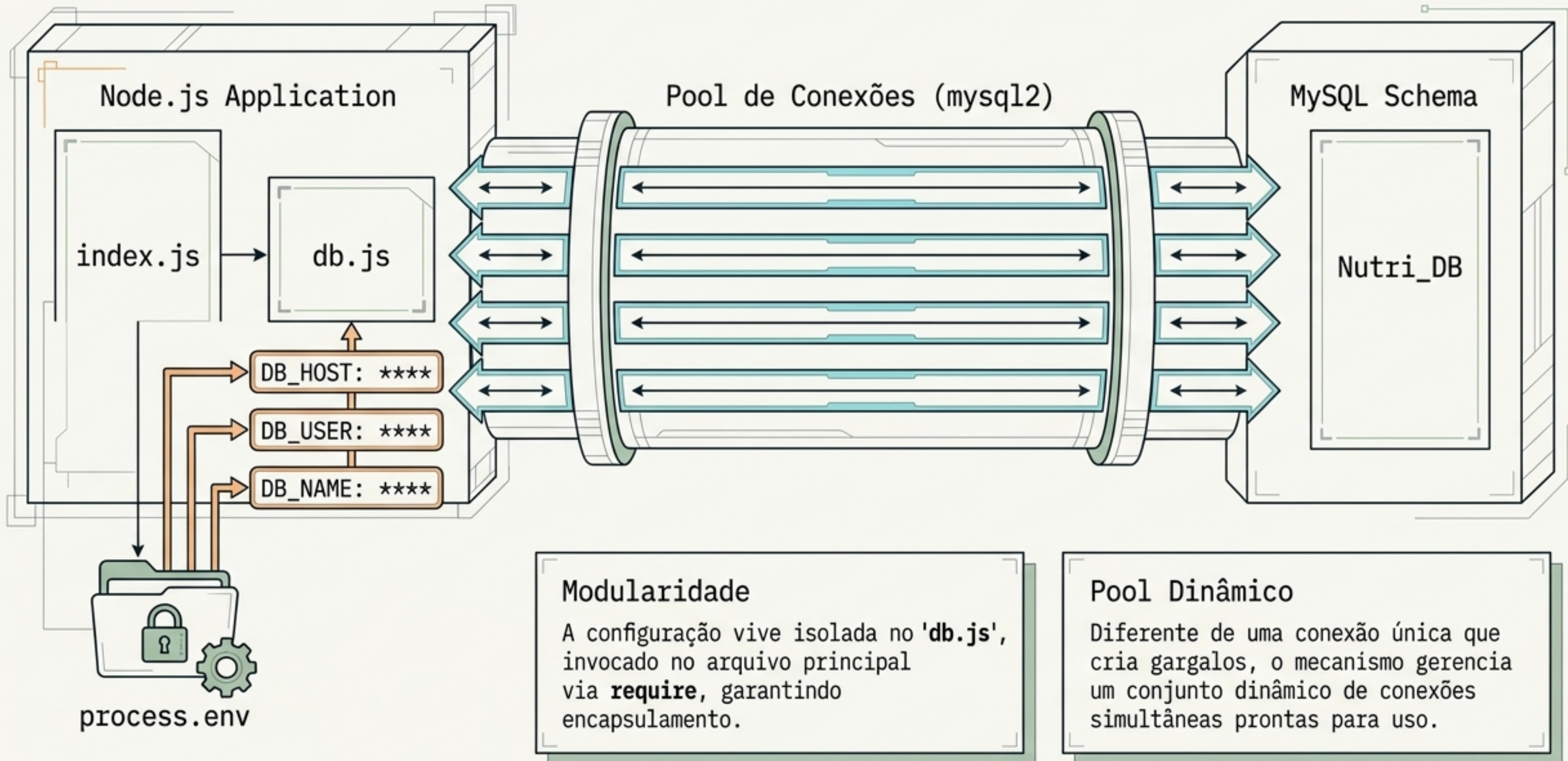
Instanciando a porta de entrada

index.js

```
const express = require('express');  
const app = express();  
app.use(express.json());
```



A ponte assíncrona de dados



0 Endpoint de listagem de pacientes

index.js

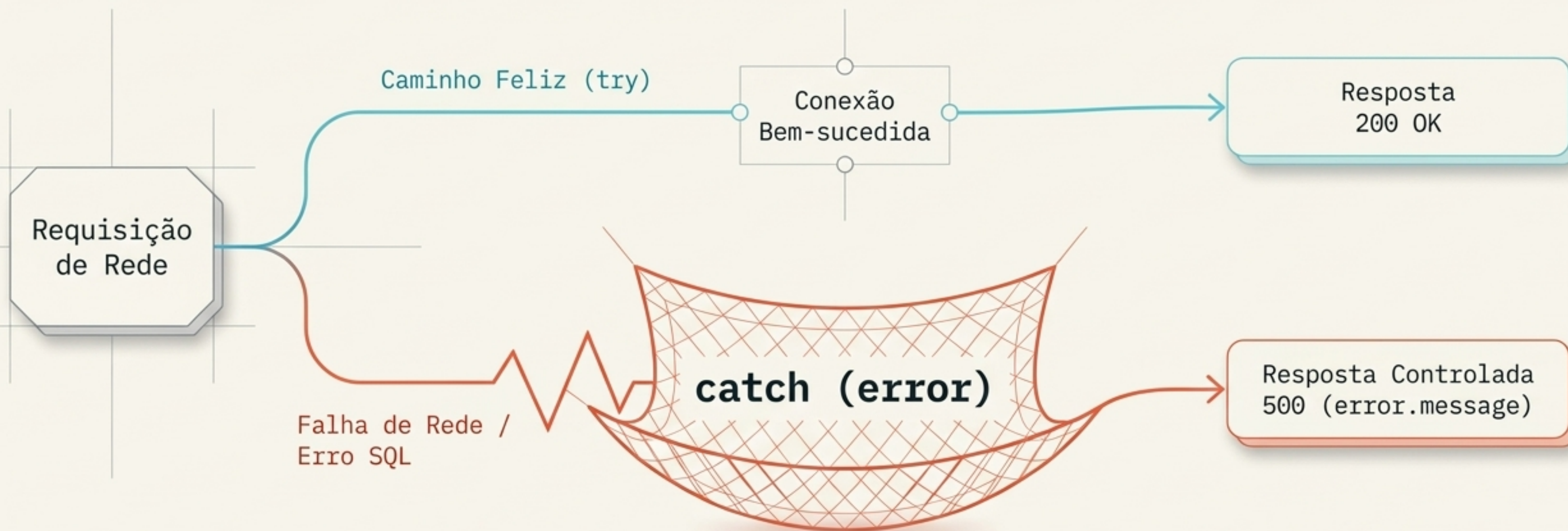
```
app.get('/relatorio', async (req, res) => {  
  const [rows] = await db.execute('SELECT * FROM clientes');  
  res.status(200).json(rows);  
});
```

① **0 Verbo:** Declaração de rota projetada exclusivamente para leitura sistemática de dados (/relatorio).

② **Pausa Inteligente:** Sinaliza ao Node para pausar apenas esta requisição até o banco responder, sem travar o restante do servidor.

③ **Destruturação Cirúrgica:** Extrai diretamente o vetor de dados reais do resultado, descartando imediatamente metadados nativos desnecessários gerados pelo MySQL.

A blindagem do servidor: Try / Catch



Sem proteção, um erro exposto paralisa o serviço e derruba a aplicação para todos os usuários. O bloco catch atua como rede de segurança, encapsulando a falha técnica e mantendo o servidor vivo.

Diagnóstico HTTP da API



[200] Sucesso

A chamada ocorreu perfeitamente. O banco retornou os dados e o processo finalizou.

Ex: `res.status(200).json(rows)`



[400] Falha do Usuário

Erro na origem. O cliente enviou parâmetros inválidos, tentou rotas inexistentes ou quebrou regras de negócio.



[500] Crise do Servidor

Onde o `catch` atua. Falha técnica no código, banco offline ou query corrompida (Internal Server Error).

A transformação universal: JSON

Base de Dados (Bruto)

	NOME	IMC	STATUS
1	Ana Silva	16.53	Abaixo do peso
2	Maria Santos	22.66	Peso normal

`res.json(rows)`

Formato Universal (JSON)

```
[  
  {  
    'nome': 'Ana Silva',  
    'imc': 16.53,  
    'status': 'Abaixo do peso'  
  },  
  {  
    'nome': 'Maria Santos',  
    'imc': 22.66,  
    'status': 'Peso normal'  
  }  
]
```

O **empacotador automático** converte instantaneamente registros rígidos do SQL para a linguagem universal da internet, pronta para ser consumida sem esforço por Front-ends, Mobile e Assistentes Virtuais.

O ciclo funcional e a próxima evolução

